

Big Data Analytics Technologies - Technical Report

Overview

This technical report summarizes the implementation and deployment details for the experimental data engineering tasks completed as part of this study. The project establishes containerized data pipelines covering time-series databases, real-time stream processing, distributed batch processing, graph databases, and theoretical data governance. While the services are containerized, they are primarily single-instance deployments intended for localized verification rather than production-scale, distributed high-availability environments. Our ingestion metrics report execution-specific throughput, though controlled latency measurements, scaling comparisons, or failover tests were outside the current scope.

Verification Run and Final State

Tasks 1–4 were reverified locally on 29 August 2026 using Docker Desktop 29.7.2, Docker Compose 5.4.0, WSL 2.7.12, 12 logical CPUs, and approximately 7.57 GiB of Docker memory. The stacks were run sequentially to avoid memory contention. Generated data, JARs, event logs, database files, metrics, and query results remain ignored by Git.

Task	Result	Fresh verification evidence	State after verification
1 — InfluxDB	PASS	96,453 rows processed; 96,429 stored temperature points; all three Flux operations passed	InfluxDB stopped; bind-mounted database retained
2 — Kafka/Flink	PASS	Three Kafka partitions, one three-slot TaskManager, RUNNING job, and completed live 15-minute sensor totals	Containers/network removed after verification; JAR/images retained
3 — Spark	PASS	7,600,595 edges, 50 ranked rows, two workers, non-zero shuffle, History Server entry	Spark services stopped; dataset/output/metrics/events retained
4 — Neo4j	PASS	5,000 relationships, three PROFILE analyses, 23 neighbours, six-hop path	Neo4j stopped cleanly; generated CSV/results retained

Task 1: Distributed Time-Series Data Management using InfluxDB

Objective: Design, provision, and evaluate a high-throughput time-series database architecture using InfluxDB inside a containerized topology, with an emphasis on time-series schema structuring, bucket retention optimization, and functional metric querying.

Architecture & Deployment

The system was deployed using a single autonomous InfluxDB 2.7.12 container and configured via Docker Compose (`task1-influxdb/docker-compose.yml`). The environment bootstrapped organization and bucket structures upon initialization using hardcoded credentials.

Data Ingestion

A Python ingestion script (`task1-influxdb/ingest/ingest.py`) parses the public Weather in Szeged CSV dataset. It translates records into the InfluxDB Line Protocol, converts the original timezone-aware timestamps to UTC, and pushes the data into the database. The importer processes 96,453 source rows, leaving 96,429 distinct temperature points after duplicate point

identities are overwritten, and reports execution-specific pipeline and database-write throughput.

Analytical Querying

Three Flux scripts were authored to perform historical and scheduled analytical operations:

1. **Sliding Hourly Average** (`01_sliding_hourly_average.flux`): Computes a one-hour temperature mean every 15 minutes.
2. **Anomaly Detection** (`02_two_sigma_anomalies.flux`): Captures climate observations more than two population standard deviations from the dataset mean.
3. **Continuous Downsampling** (`03_continuous_downsample.flux`): Summarizes the preceding hour of data into an auxiliary bucket that retains data for 30 days. Because it only queries the last hour using `range(start: -task.every)` and the source dataset ends in 2016, it does not backfill historical data.

Fresh Verification Results

Check	Result
Source rows processed	96,453
Distinct stored <code>temperature_c</code> points	96,429
Sliding one-hour averages at 15-minute intervals	385,716
Observations outside strict two-sigma limits	3,328
Synchronous InfluxDB write throughput	68,883 records/s over 1.400 s
End-to-end download/parse/write throughput	6,216 records/s over 15.517 s
Sliding-average Flux execution time	15.728 s
Two-sigma Flux execution time	3.534 s
Downsample Flux execution time	0.338 s
Downsample task	<code>climate-hourly-downsample-30d</code> , active every 1 h
Auxiliary bucket retention	720 h (30 days)

The rerun was idempotent: importing the same timestamp/tag identities did not increase the 96,429-point count. Throughput and elapsed time are observations from this host, not

controlled scalability benchmarks.

The Data Explorer interface displays a table of weather data. The table has columns: `_start`, `_stop`, `_value`, `_field`, `_measurement`, and `location`. The data rows show timestamps from 2006-01-01 12:00:00 GMT-6 to 2017-01-01 11:00:00 GMT+5:30, with values for temperature in Celsius, weather conditions, and a 'Staged' location tag.

On the left, the 'Table' view is selected. The 'Formatting' section includes options for 'Default Sort Field' (Choose a sort field), 'Time Format' (YYYY-MM-DD HH:mm:ss), and 'Decimal Places' (set to 2). The 'Colorized Thresholds' section has a '+ ADD A THRESHOLD' button and a color selection (Blue/White). The 'Colorization' section has 'BACKGROUND' and 'TEXT' tabs. The 'Column Settings' section includes 'First Column' (Scroll with table/Fixed) and 'Table Columns' (a list of columns with checkboxes).

On the right, the 'Query 1' section shows a 'FROM' clause with 'climate_30d', 'climate_raw', and '_monitoring' buckets. The 'Filter' section has three filters: '_measurement' (set to 'weather'), '_field' (set to 'temperature_c'), and 'location' (set to 'Staged'). The 'View Raw Data' button is visible. The 'SCRIPT EDITOR' section has a 'SUBMIT' button. The 'WINDOW PERIOD' section has 'CUSTOM' and 'AUTO' tabs, with 'CUSTOM' selected and a window of 'auto (1w4d3h55m55s)'.

The Tasks interface shows a list of tasks. The task 'climate-hourly-downsample-30d' is highlighted. It is 'Active' and 'Last completed at 2026-08-29T18:00:00Z'. It is 'Scheduled to run every 1h' and has an ID of '113788b9966d000'. There is an 'Add a label' button.

The Load Data interface shows a list of buckets. The bucket 'climate_30d' is highlighted. It has a 'Retention: 30 days' and an ID of 'c1e5cd9a189bceee'. There are 'Add a label', 'Add Data', and 'Settings' buttons. The bucket 'climate_raw' is also listed with 'Retention: Forever' and ID 'e5be5ebc9d66b2'. There are 'Add a label', 'Add Data', and 'Settings' buttons. The bucket '_monitoring' is listed with 'System Bucket', 'Retention: 7 days', and ID '8bd7a263e89e0f30'. The bucket '_tasks' is listed with 'System Bucket', 'Retention: 3 days', and ID '566f066f952f2a8'.

On the right, there is a 'What is a Bucket?' section with a description: 'A bucket is a named location where time series data is stored. All buckets have a Retention Policy, a duration of time that each data point persists. Here's how to write data into your bucket.'

Task 2: Real-Time Stream Ingestion & Processing with Kafka and Flink

Objective: Construct a localized stream processing infrastructure.

Architecture & Deployment

The topology is built on a Docker Compose landscape (`task2-kafka-flink/docker-compose.yml`) containing:

- One Apache Kafka broker node using `PLAINTEXT` communication and a replication factor of 1.
- One Apache Flink JobManager node.
- One Apache Flink TaskManager node.

Live Simulation

A Python producer application (`task2-kafka-flink/producer/producer.py`) sequentially replays historical Austin camera traffic-count records and pushes them into a 3-partition Kafka topic (`traffic-telemetry`) every 2 seconds.

Stateful Flink Processing

A Java-based Flink job (`com.coursework.TrafficWindowJob`) consumes the Kafka stream. It utilizes a `Bounded-OutOfOrderness` watermarking strategy with a 10-second tolerance for out-of-sequence events. Records arriving behind the watermark are dropped by default, as no `allowedLateness` or late-data side output is configured. The pipeline employs a 15-minute tumbling event-time window to produce non-overlapping vehicle-count totals per sensor. The retained TaskManager evidence shows completed windows for sensor `6171` , including a visible total of 279 vehicles.

Fresh Verification Results

Check	Result
Shaded Flink JAR size	20,105,907 bytes
Kafka topic	traffic-telemetry
Kafka partitions / replication factor	3 / 1
Flink TaskManagers / total slots	1 / 3
Submitted job state	RUNNING
Dashboard job ID	49452853a21a169172b51c412f8c22bc
Window operator records received	338
Visible output sensor	6171
Example visible 15-minute total	279

The dashboard evidence records job `49452853a21a169172b51c412f8c22bc` in the **RUNNING** state with parallelism three and 338 records received by the window operator. The retained TaskManager log screenshot shows structured JSON totals whose window start and end timestamps differ by 900,000 milliseconds, confirming completed 15-minute event-time windows. These screenshots were captured during repeated verification runs and therefore document the deployed topology and observable output without claiming that every figure belongs to one job execution. The Task 2 containers and container-local Kafka data were removed after verification; the rebuilt JAR and Docker images remain cached.

The screenshot displays the Apache Flink Web Dashboard interface. The left sidebar contains navigation links: Overview (selected), Jobs, Task Managers, Job Manager, and Submit New Job. The main content area shows the following information:

- Version:** 1.20.3, **Commit:** 01e3a6d @ 2025-09-20T02:22:05+02:00, **Message:** 0
- Available Task Slots:** 0 (Total Task Slots: 3, Task Managers: 1)
- Running Jobs:** 1 (Finished: 0, Canceled: 0, Failed: 0)
- Running Job List:**

Job Name	Start Time	Duration	End Time	Tasks	Status
Austin traffic telemetry: 15-minute tumbling sensor totals	2026-08-30 00:35:40.280	18m 42s 655ms	-	6 / 6	RUNNING
- Completed Job List:** (Empty table)

Apache Flink Web Dashboard

localhost8081/#job/running/49452853a21a169172b51c412f8c22bc/overview

OverviewJobsRunning JobsCompleted JobsTask ManagersJob ManagerSubmit New Job

Version: 1.20.3Commit: 01e3a6d @ 2025-09-20T02:22:05+02:00Message:

Austin traffic telemetry: 15-minute tumbling sensor totalsCancel Job

Job ID	49452853a21a169172b51c412f8c22bc	Job State	RUNNING2	Actions	Job Manager Log
Job Type	STREAMING	Start Time	2026-08-30 00:35:40.280	Duration	21m 26s 975ms

OverviewExceptionsData SkewTimeLineCheckpointsConfiguration

Source: traffic-telemetry-source
-> parse-structured-traffic-json
-> ten-second-bounded-out-of-orderness
Parallelism: 3
Backpressured (max): 0%
Busy (max): 0%
Data Skew: 0%

HASH

fifteen-minute-tumbling-total-by-sensor -> Sink: print-fifteen-minute-sensor-totals
Parallelism: 3
Backpressured (max): 0%
Busy (max): 0%
Data Skew: 100%
Low Watermark: 1496291389999

Name	Status	Bytes Received	Records Received	Bytes Sent	Tasks
Source: traffic-telemetry-source -> parse-structured-traffic-json -> ten-second-bounded-out-of-orderness	RUNNING	0 B	0	0 B	3
fifteen-minute-tumbling-total-by-sensor -> Sink: print-fifteen-minute-sensor-totals	RUNNING	13.4 KB	338	0 B	3

Apache Flink Web Dashboard

localhost8081/#task-manager/flink-taskmanager:6122-2c1e3a/top

OverviewJobsTask ManagersJob ManagerSubmit New Job

Version: 1.20.3Commit: 01e3a6d @ 2025-09-20T02:22:05+02:00Message:

flink-taskmanager:6122-2c1e3a

Path	pekko.tcp://flink-taskmanager:6122/user/rpc/taskmanager_0	Free/All Slots	0 / 3
Last Heartbeat	2026-08-30 01:05:05	Data Port	45489
Physical Memory	7.57 GB	CPU Cores	12
		Flink Managed Memory	512 MB

MetricsLogsStdoutLog ListThread DumpProfiler

621

2026-08-29 19:05:41,775 INFO org.apache.kafka.clients.Metadata [1] - [Consumer clientId=traffic-window-totals-1, groupId=traffic-window-totals-1, groupInstanceId=0] Resetting the last seen epoch of partition traffic-telemetry-1 to 0 since the associated topicId changed from null to Otrc40589a0TEFP2M0tg

622

2026-08-29 19:05:41,775 INFO org.apache.kafka.clients.Metadata [1] - [Consumer clientId=traffic-window-totals-2, groupId=traffic-window-totals-2, groupInstanceId=0] Resetting the last seen epoch of partition traffic-telemetry-2 to 0 since the associated topicId changed from null to Otrc40589a0TEFP2M0tg

623

2026-08-29 19:05:41,775 INFO org.apache.kafka.clients.Metadata [1] - [Consumer clientId=traffic-window-totals-0, groupId=traffic-window-totals-0, groupInstanceId=0] Resetting the last seen epoch of partition traffic-telemetry-0 to 0 since the associated topicId changed from null to Otrc40589a0TEFP2M0tg

624

2026-08-29 19:05:41,776 INFO org.apache.kafka.clients.Metadata [1] - [Consumer clientId=traffic-window-totals-1, groupId=traffic-window-totals-1, groupInstanceId=0] Resetting the last seen epoch of partition traffic-telemetry-1 to 0 since the associated topicId changed from null to Otrc40589a0TEFP2M0tg

625

2026-08-29 19:05:41,776 INFO org.apache.kafka.clients.Metadata [1] - [Consumer clientId=traffic-window-totals-2, groupId=traffic-window-totals-2, groupInstanceId=0] Resetting the last seen epoch of partition traffic-telemetry-2 to 0 since the associated topicId changed from null to Otrc40589a0TEFP2M0tg

626

2026-08-29 19:05:41,779 INFO org.apache.kafka.clients.Metadata [1] - [Consumer clientId=traffic-window-totals-0, groupId=traffic-window-totals-0, groupInstanceId=0] Resetting the last seen epoch of partition traffic-telemetry-0 to 0 since the associated topicId changed from null to Otrc40589a0TEFP2M0tg

627

2026-08-29 19:05:41,779 INFO org.apache.kafka.clients.Metadata [1] - [Consumer clientId=traffic-window-totals-1, groupId=traffic-window-totals-1, groupInstanceId=0] Resetting the last seen epoch of partition traffic-telemetry-1 to 0 since the associated topicId changed from null to Otrc40589a0TEFP2M0tg

628

2026-08-29 19:05:41,788 INFO org.apache.kafka.clients.Metadata [1] - [Consumer clientId=traffic-window-totals-2, groupId=traffic-window-totals-2, groupInstanceId=0] Resetting the last seen epoch of partition traffic-telemetry-2 to 0 since the associated topicId changed from null to Otrc40589a0TEFP2M0tg

629

2026-08-29 19:05:41,789 INFO org.apache.kafka.clients.Metadata [1] - [Consumer clientId=traffic-window-totals-0, groupId=traffic-window-totals-0, groupInstanceId=0] Cluster ID: 41g3hNt-eMCK--X86w

630

2026-08-29 19:05:41,789 INFO org.apache.kafka.clients.Metadata [1] - [Consumer clientId=traffic-window-totals-1, groupId=traffic-window-totals-1, groupInstanceId=0] Cluster ID: 41g3hNt-eMCK--X86w

631

2026-08-29 19:05:41,789 INFO org.apache.kafka.clients.Metadata [1] - [Consumer clientId=traffic-window-totals-2, groupId=traffic-window-totals-2, groupInstanceId=0] Cluster ID: 41g3hNt-eMCK--X86w

632

2026-08-29 19:05:41,833 INFO org.apache.kafka.clients.consumer.internals.SubscriptionState [1] - [Consumer clientId=traffic-window-totals-1, groupId=traffic-window-totals-1, groupInstanceId=0] Resetting offset for partition traffic-telemetry-1 to position FetchPosition[offset=0, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch[leader=Optional[kafka:0892 (id: 1 rack: null)], epoch=0]]

633

2026-08-29 19:05:41,833 INFO org.apache.kafka.clients.consumer.internals.SubscriptionState [1] - [Consumer clientId=traffic-window-totals-2, groupId=traffic-window-totals-2, groupInstanceId=0] Resetting offset for partition traffic-telemetry-2 to position FetchPosition[offset=0, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch[leader=Optional[kafka:0892 (id: 1 rack: null)], epoch=0]]

634

2026-08-29 19:05:41,834 INFO org.apache.kafka.clients.consumer.internals.SubscriptionState [1] - [Consumer clientId=traffic-window-totals-0, groupId=traffic-window-totals-0, groupInstanceId=0] Resetting offset for partition traffic-telemetry-0 to position FetchPosition[offset=0, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch[leader=Optional[kafka:0892 (id: 1 rack: null)], epoch=0]]

635

2026-08-29 19:16:41,863 INFO org.apache.kafka.clients.NetworkClient [1] - [Consumer clientId=traffic-window-totals-1, groupId=traffic-window-totals-1, groupInstanceId=0] Node -1 disconnected.

636

2026-08-29 19:16:41,863 INFO org.apache.kafka.clients.NetworkClient [1] - [Consumer clientId=traffic-window-totals-2, groupId=traffic-window-totals-2, groupInstanceId=0] Node -1 disconnected.

637

2026-08-29 19:16:41,863 INFO org.apache.kafka.clients.NetworkClient [1] - [Consumer clientId=traffic-window-totals-0, groupId=traffic-window-totals-0, groupInstanceId=0] Node -1 disconnected.

638

2026-08-29 19:16:59,790 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496275200000, "window_end_ms": 1496276100000, "vehicle_count_total": 1795}

639

2026-08-29 19:17:41,860 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496275200000, "window_end_ms": 1496276100000, "vehicle_count_total": 1803}

640

2026-08-29 19:18:23,807 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496277000000, "window_end_ms": 1496277900000, "vehicle_count_total": 2127}

641

2026-08-29 19:19:03,867 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496277900000, "window_end_ms": 1496278800000, "vehicle_count_total": 1605}

642

2026-08-29 19:19:43,892 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496278800000, "window_end_ms": 1496279700000, "vehicle_count_total": 1513}

643

2026-08-29 19:20:25,997 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496279700000, "window_end_ms": 1496280600000, "vehicle_count_total": 1502}

644

2026-08-29 19:21:06,952 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496280600000, "window_end_ms": 1496281500000, "vehicle_count_total": 1506}

645

2026-08-29 19:21:46,980 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496281500000, "window_end_ms": 1496282400000, "vehicle_count_total": 1543}

646

2026-08-29 19:22:18,257 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496282400000, "window_end_ms": 1496283300000, "vehicle_count_total": 1503}

647

2026-08-29 19:22:56,307 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496283300000, "window_end_ms": 1496284200000, "vehicle_count_total": 1760}

648

2026-08-29 19:23:32,952 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496284200000, "window_end_ms": 1496285100000, "vehicle_count_total": 1558}

649

2026-08-29 19:24:04,291 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496285100000, "window_end_ms": 1496286000000, "vehicle_count_total": 1511}

650

2026-08-29 19:24:36,542 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496286000000, "window_end_ms": 1496286900000, "vehicle_count_total": 1592}

651

2026-08-29 19:25:10,729 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496286900000, "window_end_ms": 1496287800000, "vehicle_count_total": 1558}

652

2026-08-29 19:25:42,535 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496287800000, "window_end_ms": 1496288700000, "vehicle_count_total": 1623}

653

2026-08-29 19:26:18,477 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496288700000, "window_end_ms": 1496289600000, "vehicle_count_total": 1521}

654

2026-08-29 19:26:50,512 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496289600000, "window_end_ms": 1496290500000, "vehicle_count_total": 1529}

655

2026-08-29 19:27:00,540 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496290500000, "window_end_ms": 1496291400000, "vehicle_count_total": 1531}

656

2026-08-29 19:27:30,786 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496291400000, "window_end_ms": 1496292300000, "vehicle_count_total": 1571}

657

2026-08-29 19:27:50,729 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496292300000, "window_end_ms": 1496293200000, "vehicle_count_total": 1413}

658

2026-08-29 19:28:34,786 INFO org.apache.flink.runtime.taskexecutor.SystemOutRedirectorImpl [1] - > {"sensor_id": "6171", "window_start_ms": 1496293200000, "window_end_ms": 1496294100000, "vehicle_count_total": 1371}

659

Task 3: Scalable Data Analytics with Apache Spark

Objective: Handle large-scale graph-data aggregation over Spark compute nodes.

Architecture & Deployment

A Spark cluster was orchestrated via Docker Compose (`task3-spark/docker-compose.yml`). The setup includes one Master node and two independent Worker nodes, all running on a single host. Resource constraints were applied (e.g., maximum compute cores and RAM allocation).

PySpark Job Implementation

The data processing logic (`task3-spark/apps/analyze_graph.py`) processes the 7,600,595-edge SNAP network text dataset. The script parses the text into Spark DataFrames, computes one in-degree count per destination vertex, and orders those counts to return the top 50 nodes.

Metrics Tracking

The job was submitted using `spark-submit` . Captured metrics (`metrics/execution-metrics.json`) link SQL executions to jobs, stages, and RDD lineage; retain the initial and adaptive Spark SQL physical plans as structured operator DAGs; and record stage durations, per-worker allocation, and shuffle read/write bytes. The retained metrics record two workers, 22 RDD nodes with 21 lineage edges, two SQL executions with 21 initial operators, and 5,652,839 bytes each of total shuffle read and shuffle write.

Fresh Verification Results

Check	Result
Fresh application ID	app-20260829194102-0000
Parsed directed edges	7,600,595
Ranked output rows	50
Highest-ranked destination / in-degree	438238 / 84208
Live workers	2
Resources per worker	2 cores / 2048 MiB
Completed jobs	5
Stage nodes / dependency edges	7 / 2
RDD nodes / lineage edges	22 / 21
SQL executions / initial operators	2 / 21
Adaptive SQL plan updates	6
Total shuffle read / write	5,652,839 / 5,652,839 bytes

Both workers performed work: the fresh metrics assigned eight tasks to `spark-worker-2` and nine tasks to `spark-worker-1`. The Master API reported exactly two `ALIVE` workers, and the History Server exposed the completed fresh application before the stack was stopped.

Spark Master at spark://spark-master:7077

localhost:8080

Chat

Spark

3.5.8

Spark Master at spark://spark-master:7077

URL: spark://spark-master:7077

Alive Workers: 2

Cores in use: 4 Total, 0 Used

Memory in use: 4.0 GiB Total, 0.0 B Used

Resources in use:

Applications: 0 Running, 1 Completed

Drivers: 0 Running, 0 Completed

Status: ALIVE

Workers (2)

Worker Id	Address	State	Cores	Memory	Resources
worker-20260829194047-spark-worker-1-38447	spark-worker-1:38447	ALIVE	2 (0 Used)	2.0 GiB (0.0 B Used)	
worker-20260829194048-spark-worker-2-44967	spark-worker-2:44967	ALIVE	2 (0 Used)	2.0 GiB (0.0 B Used)	

Running Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Completed Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260829194102-0000	WebBerkStanInDegreeAnalysis	4	1024.0 MiB		2026/08/29 19:41:02	root	FINISHED	18 s

History Server

localhost:18080

spark 3.5.8

History Server

Event log directory: file:///opt/spark-events

Last updated: 2026-08-30 01:16:05

Client local time zone: Asia/Colombo

Search:

Version	App ID	App Name	Started	Completed	Duration	Spark User	Last Updated	Event Log
3.5.8	app-20260829194102-0000	WebBerkStaninDegreeAnalysis	2026-08-30 01:11:02	2026-08-30 01:11:21	19 s	root	2026-08-30 01:11:21	Download

WebBerkStaninDegreeAnalysis - L X

localhost:18080/history/app-20260829194102-0000/stages/stage?id=6&attempt=0

spark 3.5.8

Jobs Stages Storage Environment Executors SQL / DataFrame

WebBerkStaninDegreeAnalysis application UI

Details for Stage 6 (Attempt 0)

Resource Profile Id: 0

Total Time Across All Tasks: 2 s

Locality Level Summary: Node local: 4

Shuffle Read Size / Records: 5.4 MiB / 645473

Associated Job Ids: 4

DAG Visualization

Stage 6

AKEShuffleRead

ShuffleReadRDD [18] [Unordered]

collect at /opt/coursework/apps/analyze_graph.py#66

WholeStageCodegen (2)

MapPartitionsRDD [19] [Unordered]

collect at /opt/coursework/apps/analyze_graph.py#66

TakeOrderedAndProject

MapPartitionsRDD [20] [Unordered]

collect at /opt/coursework/apps/analyze_graph.py#66

MapPartitionsRDD [21] [Unordered]

collect at /opt/coursework/apps/analyze_graph.py#66

Show Additional Metrics

Event Timeline

Summary Metrics for 4 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.4 s	0.5 s	0.5 s	0.5 s	0.5 s
GC Time	4.0 ms	4.0 ms	5.0 ms	5.0 ms	5.0 ms
Shuffle Read Size / Records	1.3 MiB / 158075	1.3 MiB / 158413	1.4 MiB / 161579	1.4 MiB / 167406	1.4 MiB / 167406

Aggregated Metrics by Executor

Tasks (4)

Show 20 entries

Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Read Size / Records	Errors
0	13	0	SUCCESS	NODE_LOCAL	0	spark-worker-2	stdout stderr	2026-08-30 01:11:20	0.5 s	5.0 ms	1.4 MiB / 161579	
1	14	0	SUCCESS	NODE_LOCAL	1	spark-worker-1	stdout stderr	2026-08-30 01:11:20	0.5 s	4.0 ms	1.3 MiB / 158075	
2	15	0	SUCCESS	NODE_LOCAL	0	spark-worker-2	stdout stderr	2026-08-30 01:11:20	0.5 s	5.0 ms	1.3 MiB / 158413	
3	16	0	SUCCESS	NODE_LOCAL	1	spark-worker-1	stdout stderr	2026-08-30 01:11:20	0.4 s	4.0 ms	1.4 MiB / 167406	

Showing 1 to 4 of 4 entries

Previous 1 Next

Task 4: Graph Database Engineering using Neo4j

Objective: Develop graph-native storage structures and run node lookups.

Architecture & Deployment

A single Neo4j Community Edition instance was spun up via Docker Compose (`task4-neo4j/docker-compose.yml`). The container exposed HTTP and Bolt ports without TLS.

Password authentication was enabled, but Neo4j's `/data` directory was not mounted, causing the database to disappear after the container is removed.

Graph Ingestion

Using Cypher's `LOAD CSV` mechanism and `MERGE` statements (`task4-neo4j/cypher/import_patents.cypher`), the first 5,000 directed citation edges (with citing patent IDs covering `3858241-3859217`) from the SNAP patent dataset were imported into the graph.

Network Analytics

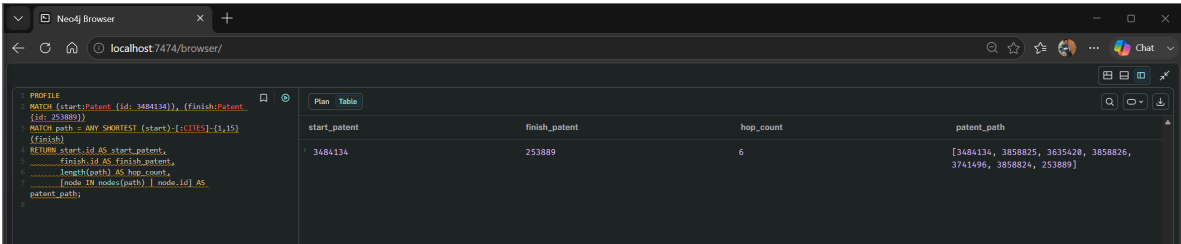
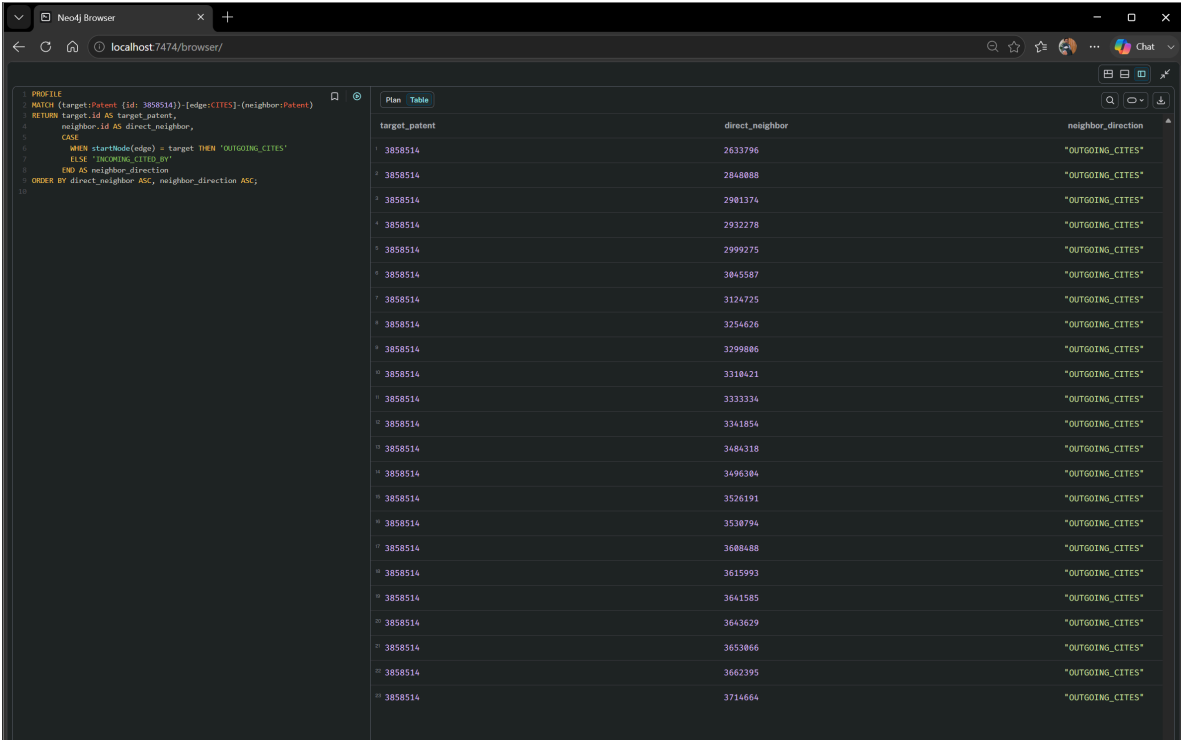
Cypher queries (`task4-neo4j/cypher/analysis_queries.cypher`) were formulated to analyze the subset graph structure:

1. A one-hop, direction-agnostic neighborhood query around patent `3858514` .
2. One subset in-degree ranking to discover patents with incoming citations within the subset, where the maximum observed in-degree is 4.
3. One undirected six-hop shortest path between patents `3484134` and `253889` .

Fresh Verification Results

Check	Result
Prepared/imported <code>CITES</code> relationships	5,000
Direct neighbours of patent <code>3858514</code>	23
Highest observed subset in-degree	4
Patents tied at in-degree 4	<code>3284344</code> , <code>3308054</code> , <code>3338819</code> , <code>3717571</code>
Shortest-path hop count	6
PROFILE statements/results	3
Saved analysis-output size	17,133 bytes
Neo4j Browser HTTP status	200

The verified shortest path was `[3484134, 3858825, 3635420, 3858826, 3741496, 3858824, 253889]` . The prepared subset is retained in `task4-neo4j/import/patent_edges_5000.csv` , and the three saved result/plan sections are retained in `task4-neo4j/results/query-execution.txt` . Neo4j's `/data` directory remains container-local, so removing the Neo4j container deletes the imported graph even though the prepared CSV and saved analysis output survive.



Task 5: Theoretical Foundations & Future Engineering Trends

Objective: Scoped narrative literature review on data governance and advanced computing paradigms.

This component (detailed in `task5-literature-review.md`) delivers a narrative synthesis of theoretical constructs impacting modern data engineering:

1. **Enterprise Data Governance:** Compares centralized Data Lakes with decentralized Data Mesh architectures and Data Contracts. It covers how metadata tagging aids in GDPR/CCPA compliance, acknowledging that deletion rights contain exceptions and are not always "complete," and explains that while metadata lineage tracks data flows, access/audit logs are required to determine exactly who accessed data.
2. **Emerging Frontiers:** Analyzes Quantum Machine Learning (QML) and Large Language Model (LLM)-driven data engineering cycles. It explores how LLM code-generation variability poses a distinct challenge from Spark/Flink's exactly-once requirements (which depend on checkpointing, replayable sources, and transactional or idempotent sinks). It discusses self-healing ETL as a prospective application rather than a demonstrated result in the cited review.